

Actividad 02 — Construcción de Portafolios Óptimos

Enfoque Incremental: EDA $\rightarrow$ Markowitz (Óptimo) $\rightarrow$ Simulación (Validación) $\rightarrow$ Servicing
(Manual vs Desarrollo Asistido con IA)

Sebastián Egaña Santibáñez (segana@fen.uchile.cl)
Nicolás Leiva Díaz (nleivad@fen.uchile.cl)

2026-09-21

Descargas y Enlaces

Enlaces del profesor

<https://segana.netlify.app>

<https://github.com/sebaegana>

<https://www.linkedin.com/in/sebastian-egana-santibanez/>

Introducción: El Problema

Contexto: Tienes acceso a 5 activos financieros: SPY (S&P 500), EFA (mercados desarrollados), IJS (small-cap), EEM (mercados emergentes), AGG (bonos). Deseas construir un portafolio que maximice el retorno relativo al riesgo, pero **no sabes qué pesos asignar** a cada uno.

Pregunta central: - ¿Cuál es la combinación *óptima* de pesos? - ¿Cómo *verificar* que esa solución es realmente óptima? - ¿Cómo *publicar* ese resultado para que otros lo usen?

Tu rol: seguir un pipeline de datos completo: 1. **Parte 1 — EDA:** entender los datos y correlaciones entre activos 2. **Parte 2 — Markowitz:** calcular la frontera eficiente (solución analítica exacta) 3. **Parte 3 — Simulación:** generar miles de portafolios aleatorios para validar que Markowitz es realmente óptimo 4. **Parte 4 — Comparación:** ¿cuán buena es la nube simulada vs la frontera exacta? 5. **Parte 5 — Servicing:** publicar el resultado como una página interactiva en este sitio

En cada parte, verás dos enfoques: **Manual** (escribes cada línea de código) y **Asistido con Claude Code** (acelera 5-10x).

Parte 1 — EDA: Explorando los Datos y Correlaciones

Enfoque Manual (Tradicional)

Sin IA, siguiendo la estructura clásica:

Paso 1: Cargar librerías y datos

```
library(readr)
library(tidyverse)
library(skimr)
library(corrplot)
library(kableExtra)

portafolio_retornos <- read_csv("data/portafolio_retornos.csv")

head(portafolio_retornos)
str(portafolio_retornos)
```

Tiempo: ~3 minutos.

Paso 2: Estadística descriptiva

```
summary(portafolio_retornos |> select(-fecha))
skim(portafolio_retornos |> select(-fecha))

stats_por_activo <- portafolio_retornos |>
  select(-fecha) |>
  summarise(across(everything(),
    list(
      media = ~mean(., na.rm = TRUE),
      sd = ~sd(., na.rm = TRUE),
      min = ~min(., na.rm = TRUE),
      max = ~max(., na.rm = TRUE),
      n = ~n()
    )
  ),
```

```

    .names = "{.col}_{.fn}"
  ))

print(stats_por_activo)

```

Tiempo: ~10 minutos.

Paso 3: Visualizaciones

```

portafolio_retornos |>
  select(-fecha) |>
  pivot_longer(everything(), names_to = "activo", values_to = "retorno") |>
  ggplot(aes(x = retorno, fill = activo)) +
  geom_histogram(alpha = 0.45, binwidth = 0.01) +
  facet_wrap(~activo, scales = "free") +
  theme_minimal() +
  labs(title = "Distribución de Retornos Mensuales (2012-2021)",
       x = "Retorno", y = "Frecuencia")

portafolio_retornos |>
  mutate(fecha = as.Date(fecha)) |>
  pivot_longer(-fecha, names_to = "activo", values_to = "retorno") |>
  ggplot(aes(x = fecha, y = retorno, color = activo)) +
  geom_line(alpha = 0.7) +
  facet_wrap(~activo, scales = "free_y") +
  theme_minimal() +
  labs(title = "Retornos Mensuales en el Tiempo",
       x = "Fecha", y = "Retorno")

```

Tiempo: ~12 minutos.

Paso 4: Matriz de correlación y covarianza

```

retornos_matrix <- portafolio_retornos |>
  select(-fecha) |>
  as.matrix()

correlacion <- cor(retornos_matrix)
corrplot(correlacion, method = "circle", type = "upper",
         title = "Matriz de Correlaciones entre Activos",
         mar = c(0, 0, 2, 0))

covarianza <- cov(retornos_matrix)
print("Matriz de Covarianza:")
print(round(covarianza, 6))

correlacion_df <- as.data.frame(correlacion) |>

```

```
rownames_to_column("activo1") |>
pivot_longer(-activo1, names_to = "activo2", values_to = "corr") |>
filter(activo1 < activo2) |>
arrange(desc(abs(corr)))

kable(correlacion_df, digits = 3, caption = "Correlaciones entre Activos")
```

Tiempo: ~15 minutos.

Total Parte 1 Manual: ~40 minutos.

Enfoque Asistido con Claude Code

Prompt:

“Necesito un EDA completo de un portafolio de 5 activos desde portafolio_retornos.csv:
 1. Estadística descriptiva (media, sd, quantiles, valores faltantes) 2. Histogramas de retornos por activo 3. Series temporales de retornos 4. Matriz de correlación y covarianza con visualización”

Resultado (minutos):

Claude Code genera un script optimizado con tidyverse + corrplot + ggplot2.

Tiempo: ~5 minutos (lectura + ajustes).

Comparación: Manual vs IA (Parte 1)

Aspecto	Manual	Con IA
Tiempo	~40 min	~5 min
Cobertura	Buena, incompleta	Exhaustiva
Debugging	Manual, prueba-error	IA propone soluciones
Calidad código	Iterativo	Producción

Parte 2 — Markowitz: Optimización de la Frontera Eficiente

¿Por qué Markowitz primero?

Markowitz es la **solución óptima exacta** (analítica). Entender cómo se resuelve es fundamental antes de aproximarla con simulación.

Enfoque Manual

Paso 1: Preparar datos y matriz de covarianza

```
portafolio_retornos <- read_csv("data/portafolio_retornos.csv")

retornos_matrix <- portafolio_retornos |>
  select(~fecha) |>
  as.matrix()

retornos_esperados <- colMeans(retornos_matrix)
cov_matrix <- cov(retornos_matrix)
tasa_libre_riesgo <- 0.02 / 12
```

Tiempo: ~5 minutos.

Paso 2: Formular el problema de Markowitz

La frontera eficiente resuelve:

$$\min_w [w^T \Sigma w]$$

sujeto a: - $w^T r = r_p$ (retorno objetivo) - $\sum w_i = 1$ (pesos suman 1) - $w_i \geq 0$ (long-only, sin venta corta)

Paso 3: Instalar quadprog y resolver

```
# install.packages("quadprog")
library(quadprog)
```

```

n_activos <- ncol(retornos_matrix)
retornos_objetivo <- seq(0.08, 0.20, length.out = 50) / 12 # mensuales

frontera_markowitz <- data.frame(
  retorno = numeric(length(retornos_objetivo)),
  volatilidad = numeric(length(retornos_objetivo)),
  pesos = matrix(NA, nrow = length(retornos_objetivo), ncol = n_activos)
)

for (i in seq_along(retornos_objetivo)) {
  r_obj <- retornos_objetivo[i]

  # Formulación quadprog
  H <- 2 * cov_matrix
  f <- rep(0, n_activos)

  # Restricciones: sum(w)=1, w'*r=r_obj, w>=0
  A <- cbind(
    rep(1, n_activos),      # suma = 1
    retornos_esperados,     # retorno objetivo
    diag(n_activos)        # w >= 0
  )

  b0 <- c(1, r_obj, rep(0, n_activos))

  resultado <- solve.QP(H, f, t(A), b0, meq = 2)

  w_opt <- resultado$solution
  vol_opt <- sqrt(t(w_opt) %*% cov_matrix %*% w_opt) * sqrt(12)

  frontera_markowitz$retorno[i] <- r_obj * 12
  frontera_markowitz$volatilidad[i] <- vol_opt
  frontera_markowitz$pesos[i, ] <- w_opt
}

# Identificar portafolio tangente (máx Sharpe)
frontera_markowitz$sharpe <- (frontera_markowitz$retorno - 0.02) / frontera_markowitz$volatilidad
idx_tangente <- which.max(frontera_markowitz$sharpe)
portafolio_tangente <- frontera_markowitz[idx_tangente, ]

```

Tiempo: ~20 minutos (comprensión de quadprog, debugging de restricciones).

Paso 4: Visualizar la frontera exacta

```

frontera_markowitz |>
  ggplot(aes(x = volatilidad, y = retorno)) +
  geom_line(size = 1.2, color = "darkred") +

```

```
geom_point(data = portafolio_tangente, color = "red", size = 4) +  
theme_minimal() +  
labs(title = "Frontera Eficiente de Markowitz (Exacta)",  
      x = "Volatilidad Anual (%)",  
      y = "Retorno Anual Esperado (%)")
```

Tiempo: ~5 minutos.

Total Parte 2 Manual: ~30 minutos.

Enfoque Asistido con Claude Code

Prompt:

“Calcula la frontera eficiente de Markowitz usando quadprog: - Long-only (pesos entre 0 y 1, suman 1) - Barre 50 puntos de retorno objetivo entre 8 % y 20 % anual - Para cada punto, minimiza varianza sujeto al retorno objetivo - Identifica el portafolio tangente (máximo Sharpe ratio) - Grafica la frontera con el punto tangente marcado en rojo”

Resultado (minutos):

Claude Code maneja la sintaxis tediosa de `quadprog` automáticamente.

Tiempo: ~5 minutos (lectura + ajustes).

Parte 3 — Simulación: Aproximar la Frontera por Monte Carlo

¿Por qué simulación después de Markowitz?

Ahora que conoces la solución *exacta*, usamos simulación para **validar**: “¿qué tan buena es una aproximación por fuerza bruta?”

Enfoque Manual

Paso 1: Simular miles de portafolios aleatorios

```
set.seed(42)

n_sims <- 10000
n_activos <- ncol(retornos_matrix)

resultados_sim <- data.frame(
  retorno_anual = numeric(n_sims),
  volatilidad_anual = numeric(n_sims),
  sharpe_ratio = numeric(n_sims)
)

pesos_matrix <- matrix(NA, nrow = n_sims, ncol = n_activos)

for (i in 1:n_sims) {
  # Generar pesos aleatorios long-only
  w <- runif(n_activos)
  w <- w / sum(w)

  # Retorno esperado del portafolio
  ret_port <- sum(w * retornos_esperados) * 12

  # Volatilidad del portafolio
  vol_port <- sqrt(t(w) %*% cov_matrix %*% w) * sqrt(12)

  # Sharpe ratio
```

```

sharpe <- (ret_port - 0.02) / vol_port

resultados_sim$retorno_anual[i] <- ret_port
resultados_sim$volatilidad_anual[i] <- vol_port
resultados_sim$sharpe_ratio[i] <- sharpe
pesos_matrix[i, ] <- w
}

# Mejor portafolio simulado
idx_mejor <- which.max(resultados_sim$sharpe_ratio)
mejor_sim <- resultados_sim[idx_mejor, ]
mejor_pesos_sim <- pesos_matrix[idx_mejor, ]

```

Tiempo: ~15 minutos.

Paso 2: Visualizar la nube

```

resultados_sim |>
  ggplot(aes(x = volatilidad_anual, y = retorno_anual, color = sharpe_ratio)) +
  geom_point(alpha = 0.5, size = 1) +
  scale_color_viridis_c(direction = 1) +
  geom_point(data = mejor_sim, color = "red", size = 4, shape = 21, stroke = 2) +
  theme_minimal() +
  labs(title = "Frontera Eficiente Simulada (10,000 portafolios)",
        x = "Volatilidad Anual (%)",
        y = "Retorno Anual Esperado (%)",
        color = "Sharpe Ratio")

```

Tiempo: ~10 minutos.

Total Parte 3 Manual: ~25 minutos.

Enfoque Asistido con Claude Code

Prompt:

“Simula 10,000 portafolios long-only con pesos aleatorios. Para cada uno: retorno esperado (anualizado), volatilidad (anualizada), Sharpe ratio. Identifica el mejor portafolio por Sharpe. Grafica la nube coloreada por Sharpe, marcando el mejor con un punto rojo.”

Resultado (minutos):

Claude genera el loop o versión vectorizada automáticamente.

Tiempo: ~3 minutos.

Parte 4 — Comparación: ¿Cuán Buena es la Simulación?

El Gráfico Overlay: Validación Visual

Ahora superpones ambos resultados:

```
# Gráfico combinado: nube simulada + frontera exacta
ggplot() +
  geom_point(data = resultados_sim,
             aes(x = volatilidad_anual, y = retorno_anual, color = sharpe_ratio),
             alpha = 0.4, size = 1) +
  geom_line(data = frontera_markowitz,
            aes(x = volatilidad, y = retorno),
            color = "red", size = 1.5, linetype = "dashed") +
  geom_point(data = mejor_sim,
             color = "darkred", size = 4, shape = 21, stroke = 2) +
  geom_point(data = portafolio_tangente,
             color = "darkgreen", size = 4, shape = 23, stroke = 2) +
  scale_color_viridis_c(direction = 1) +
  theme_minimal() +
  labs(title = "Frontera Eficiente: Simulación (azul) vs Markowitz (roja)",
       x = "Volatilidad Anual (%)",
       y = "Retorno Anual Esperado (%)",
       color = "Sharpe (Sim)")
```

Análisis: ¿Qué Observas?

1. La nube simulada toca la frontera roja? → Depende de N (10k no es suficiente)
2. El mejor punto simulado tangente de Markowitz? → Normal, es una aproximación
3. ¿Cuán lejano está el mejor simulado del tangente? → Mide la “brecha de optimización”

Cálculo de la brecha:

```
# Distancia euclidiana entre mejor simulado y tangente de Markowitz
brecha_retorno <- abs(mejor_sim$retorno_anual - portafolio_tangente$retorno[idx_tangente])
brecha_vol <- abs(mejor_sim$volatilidad_anual - portafolio_tangente$volatilidad[idx_tangente])
```

```
brecha_sharpe <- abs(mejor_sim$sharpe_ratio - portafolio_tangente$sharpe[idx_tangente])

cat("Brecha de optimización:\n")
cat("Retorno:", brecha_retorno, "\n")
cat("Volatilidad:", brecha_vol, "\n")
cat("Sharpe:", brecha_sharpe, "\n")
cat("\nConclusión: Con 10k simulaciones, el error es de ~",
    round(brecha_sharpe / portafolio_tangente$sharpe[idx_tangente] * 100, 2), "%\n")
```

Enfoque con Claude Code

```
# Prompt: "Superpón nube simulada + frontera Markowitz.
# Marca mejor simulado en rojo, tangente en verde.
# Calcula distancia euclidiana entre ambos puntos."
```

Tiempo: ~10 minutos total.

Parte 5 — Servicing: Publicar como Página Interactiva en GitHub Pages

¿Por qué servir importa?

Hasta aquí tienes un análisis que funciona en tu máquina. **Servicing** = convertir ese análisis en una **página web interactiva** que otros puedan usar sin ejecutar código.

No necesitas servidor costoso: GitHub Pages + Quarto + Observable JS = gratis + automático.

El flujo completo (Quarto → GitHub Pages)

Paso 1: Crear archivo Quarto con Observable JS

Tu archivo `actividad_02_portafolios.qmd` ya tiene un bloque `{ojs}` (Observable JS):

```
// Parte 5: Observable JS embebido en el .qmd
// Carga datos desde CSV
portafolio_data = FileAttachment("data/portafolio_retornos.csv").csv()

// Crea 5 sliders (0-100%) para ajustar pesos de cada activo
peso_spy = Inputs.range([0, 100], {value: 20, step: 1, label: "SPY (%)"})
peso_efa = Inputs.range([0, 100], {value: 25, step: 1, label: "EFA (%)"})
// ... etc

// Calcula dinámicamente: retorno/volatilidad/Sharpe cuando mueves sliders
retorno_portfolio = sum(pesos * retornos_esperados)

// Muestra resultado en una caja HTML
html`<p>Retorno Esperado: ${ (retorno_portfolio * 100).toFixed(2) }% anual</p>`
```

Ventajas del enfoque: - Sin servidor Shiny ni backend - Solo HTML estático + JavaScript del cliente (rápido, seguro) - Funciona offline (excepto cargar CSV) - Responsive (móvil, tablet, desktop)

Paso 2: Renderizar localmente

En tu terminal:

```
# Renderiza el .qmd a HTML
quarto render clases_ia/actividad_02_portafolios.qmd --to html

# Resultado: docs/clases_ia/actividad_02_portafolios.html (con Observable JS)
```

Paso 3: Publicar en GitHub Pages (automático)

```
# Commit + push
git add clases_ia/actividad_02_portafolios.qmd
git add docs/clases_ia/actividad_02_portafolios.html
git commit -m "feat: Add interactive portfolio servicing page"
git push origin develop
# → Merge a main (después de PR)
```

Paso 4: Acceder a la página

La configuración de `_quarto.yml` ya apunta a GitHub Pages:

```
project:
  output-dir: docs # Quarto renderiza aquí
website:
  site-url: https://sebaegana.github.io/finanzas_en_r_webpage/
```

GitHub Pages automáticamente sirve archivos en `docs/` → tu página vive en:

```
https://sebaegana.github.io/finanzas_en_r_webpage/
  clases_ia/
    actividad_02_portafolios.html ← Aquí
```

¿Cómo construir la página Observable JS?

Estructura básica:

```
// 1. Cargar datos
datos = FileAttachment("data/portafolio_retornos.csv").csv()

// 2. Crear controles interactivos (sliders, botones, dropdowns)
slider_spy = Inputs.range([0, 100], {label: "SPY (%)"})
slider_efa = Inputs.range([0, 100], {label: "EFA (%)"})

// 3. Definir lógica reactiva (recalcula automáticamente cuando cambien sliders)
pesos = [slider_spy, slider_efa, ...] // Reactivo: actualiza cuando muevas sliders
retorno = suma(pesos * retornos_historicos)
sharpe = (retorno - tasa_libre) / volatilidad

// 4. Mostrar resultados
```

```
html`
<div>
  <p>Retorno: ${retorno.toFixed(2)}%</p>
  <p>Sharpe: ${sharpe.toFixed(2)}</p>
</div>
`
```

Recursos Observable JS:

- [Observable Help](#)
- [Quarto + OJS Tutorial](#)
- [D3.js para gráficos](#)

Enfoque manual vs con IA

Paso	Manual	Con IA
Diseñar estructura	15 min	2 min (describir qué quieres)
Escribir Observable JS	30 min	5 min (pedir código)
Renderizar + publicar	5 min	2 min
Total	~50 min	~9 min

Checklist para “Servicing” listo para producción

- ☐ Archivo `.qmd` con bloque `{ojs}` incluido
- ☐ Datos CSV en `clases_ia/data/` accesibles
- ☐ `quarto render` genera HTML sin errores
- ☐ Sliders/controles funcionan interactivos en el navegador
- ☐ Página responsive (prueba en móvil)
- ☐ Fecha de última actualización visible (`date: last-modified`)
- ☐ Link a la página en `README` o tabla de materiales

Ejemplo real: Este sitio

El archivo `clases_ia/actividad_02_portafolios.qmd` (que estás leyendo) es exactamente eso:

1. Fuente: `.qmd` con bloques `{ojs}`
2. Renderiza: `quarto render` → `docs/clases_ia/actividad_02_portafolios.html`
3. Publica: GitHub Pages sirve automáticamente
4. Resultado: Página interactiva en vivo

Tiempo total publicación: 3 minutos (render + push + GitHub Pages auto-deploy)

Próximo paso

Después de terminar tu análisis (Partes 1-4), **agrega un bloque {ojs}** en Parte 5 con: - Sliders para ajustar pesos del portafolio - Cálculo dinámico de retorno/volatilidad/Sharpe - Overlay gráfico de la frontera de Markowitz (opcional: usa Plotly o Observable Plot)

Luego: `quarto render + git push` = **página interactiva viva en 5 minutos**.

Tabla de Ventajas: Manual vs IA

Aspecto	Manual (Total)	Con IA (Total)
Parte 1 (EDA)	40 min	5 min
Parte 2 (Markowitz)	30 min	5 min
Parte 3 (Simulación)	25 min	3 min
Parte 4 (Comparación)	15 min	8 min
Parte 5 (Servicing)	20 min	3 min
TOTAL	~130 min	~24 min

Actividad para Estudiantes

Instrucciones Generales

Elige UNA de las 3 opciones. Todas conducen al mismo resultado: análisis completo + página publicada.

Opción A: Maximalista (Aprender escribiendo código)

Sigue todas las 5 partes escribiendo código tú mismo. Usa Claude Code solo para debugging o sugerencias puntuales.

Entregas: script R/Quarto comentado, gráficos (nube + frontera), página interactiva.

Opción B: Híbrida (Equilibrada)

1. Parte 1: Escribes EDA (10 min)
2. Parte 2: Pides a Claude que genere Markowitz (5 min lectura)
3. Parte 3: Escribes simulación con ayuda (15 min)
4. Parte 4: Análisis de brecha con Claude (10 min)
5. Parte 5: Claude genera Observable JS (5 min ajustes)

Opción C: Minimalista (Prompt-Driven)

Pídele a Claude Code el flujo completo en 1-2 prompts. Claude genera ~800 líneas en ~5 minutos. Tu rol: entender, interpretar, reflexionar.

Rúbrica de Evaluación

Criterio	Excelente	Bueno	Aceptable
EDA	Stats + 3+ gráficos + matriz de correlación clara	2 gráficos + correlación	Solo summary()
Markowitz	Frontera exacta (50+ puntos), long-only correcto, portafolio tangente identificado	20-50 puntos, correcto	Incompleto/erróneo

Criterio	Excelente	Bueno	Aceptable
Simulación	5k+ portafolios, pesos long-only correctos, nube visualizada	1-5k portafolios, correcto	<1k o errores
Comparación	Overlay claro, brecha calculada, interpretación de diferencias	Overlay presente	Ausente
Servicing	Sliders funcionales, calcula dinámicamente, responsive	Sliders básicos	Ausente
Interpretación	Explica por qué Markowitz > simulación, cuándo usar cada uno	Menciona diferencias	Mínima
Uso de IA	Estratégico (boilerplate, debugging, iteración)	Parcial	Mínimo

Entrega

1. **Script R/Quarto** comentado con las 5 partes
2. **Reporte** (HTML/PDF) con gráficos + análisis
3. **Página publicada** (GitHub Pages) con sliders interactivos
4. **Reflexión** (1 página): cómo usaste IA, brecha encontrada, lecciones aprendidas

Reflexión Final: IA como Herramienta, No Reemplazo

Usa IA para:

- Boilerplate (cargas, transformaciones estándar)
- Exploración de sintaxis (“¿cómo uso quadprog?”)
- Debugging (“¿por qué falla esta restricción?”)
- Observable JS (sintaxis compleja)
- Documentación y comentarios

NO uses IA para:

- Decisiones de estrategia (tú eliges qué validar)
 - Validación de resultados (verifica siempre que $\text{Sharpe} = (\text{ret} - \text{rf}) / \text{vol}$)
 - Comprensión (entiende el código antes de publicarlo)
 - Pensar (IA acelera ejecución, no estrategia)
-

Recursos Adicionales

[Markowitz Modern Portfolio Theory](#)

[quadprog: Quadratic Programming](#)

[Claude Code Docs](#)

[Positron IDE](#)

[Observable JS + D3](#)